

Arrays and Strings

Arrays

- **Array:** It is a collection of items of same type stored at continuous memory locations.

Syntax:

`DataType ArrayName[size_of_array];`

Example : `int arr[9];`

40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

Array Length = 9

First Index = 0

Last Index = 8

- **Basic terminologies of array**
 - **Array Index:** In an array, elements are identified by their indexes. Array index starts from 0.
 - **Array element:** Elements are items stored in an array and can be accessed by their index.
 - **Array Length:** The length of an array is determined by the number of elements it can contain.

Array Declaration

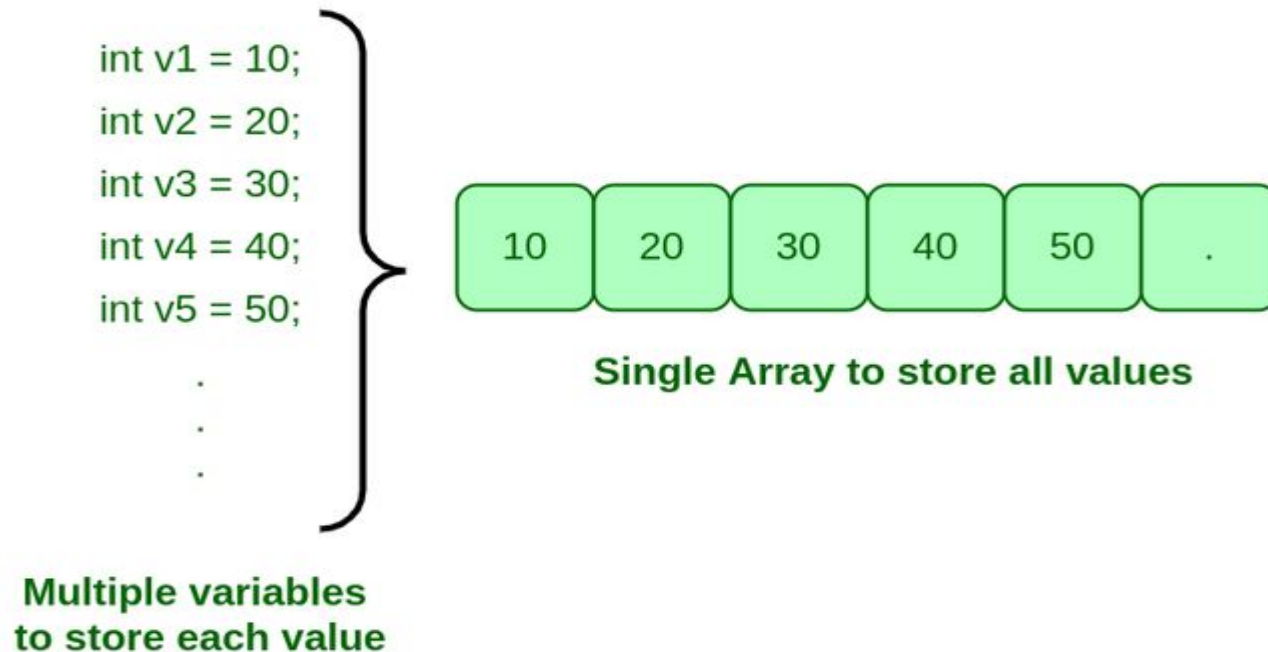
- `int arr[5];` `// This array will store integer type element`
- `char arr[10];` `// This array will store char type element`
- `float arr[20];` `// This array will store float type element`



The above declaration is **static** or **compile-time** memory allocation, which means that the array element's memory is allocated when a program is compiled. Here only a fixed size (i.e. the size that is mentioned in square brackets []) of memory will be allocated for storage

Why Array Data Structures is needed?

- Assume there is a class of five students and if we have to keep records of their marks in examination then, we can do this by declaring five variables individual and keeping track of records but what if the number of students becomes very large, it would be challenging to manipulate and maintain the data.
- What it means is that, we can use normal variables (v1, v2, v3, ..) when we have a small number of objects. But if we want to store a large number of instances, it becomes difficult to manage them with normal variables. **The idea of an array is to represent many instances in one variable..**

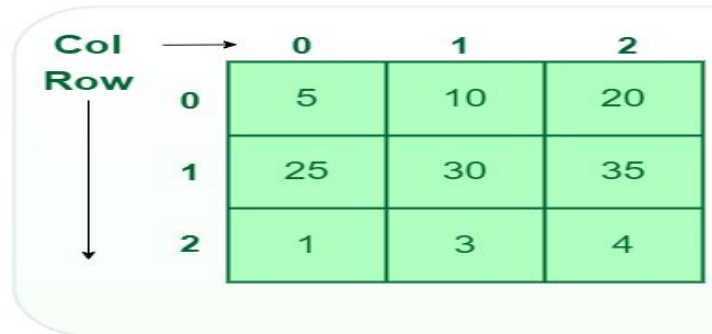


Types of arrays

- [One-dimensional array \(1-D arrays\)](#): You can imagine a 1d array as a row, where elements are stored one after another.



- Two-dimensional array: 2-D Multidimensional arrays can be considered as an array of arrays or as a matrix consisting of rows and columns.

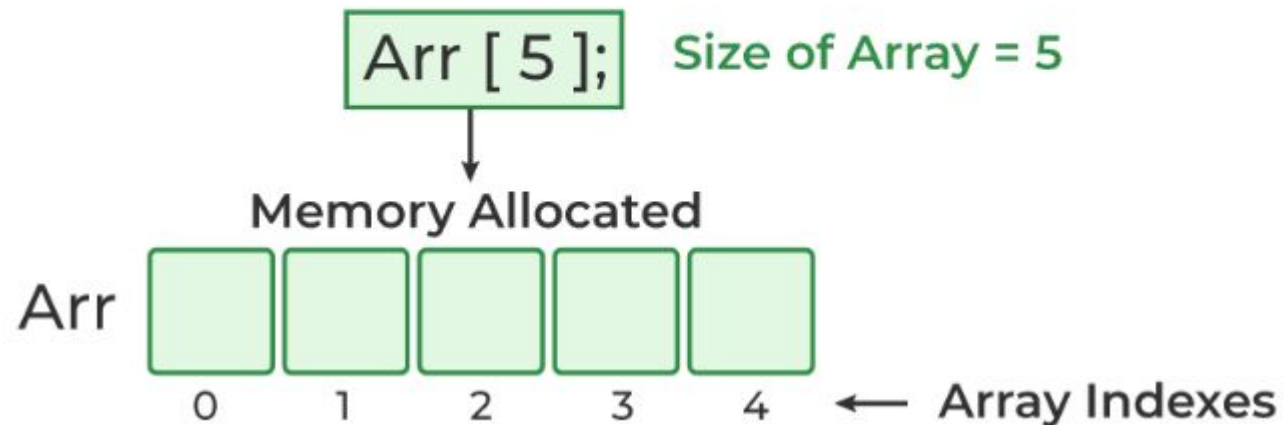


- **n-dimensional array**: A n-D [Multidimensional array](#) contains n dimensions.

Array Declaration

- `data_type array_name [size];`
or
- `data_type array_name [size1] [size2]...[sizeN];`
where N is the number of dimensions.

Array Declaration



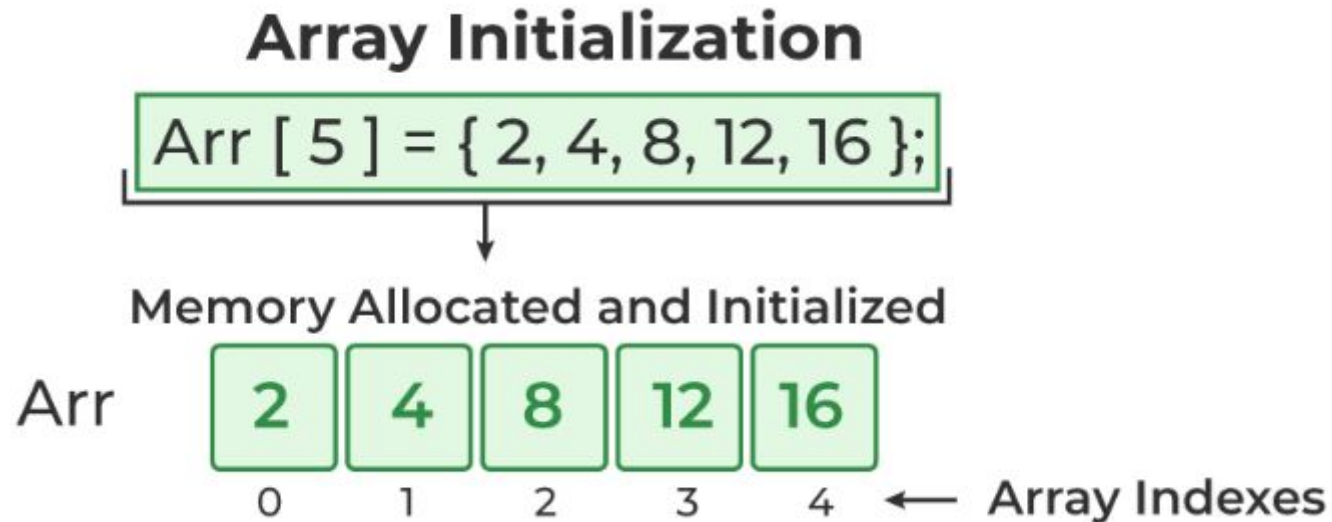
The C arrays are static in nature, i.e., they are allocated memory at the compile time.

Array Initialization

Initialization in C is the process to assign some initial value to the variable. When the array is declared or allocated memory, the elements of the array contain some garbage value. So, we need to initialize the array to some meaningful value. There are multiple ways in which we can initialize an array in C.

1. Array Initialization with Declaration

```
data_type array_name [size] = {value1, value2, ... valueN};
```



Array Initialization

2. Array Initialization with Declaration without Size

If we initialize an array using an initializer list, we can skip declaring the size of the array as the compiler can automatically deduce the size of the array in these cases.

```
data_type array_name[] = {1,2,3,4,5};
```

The size of the above arrays is 5 which is automatically deduced by the compiler.

Array Initialization

3. Array Initialization after Declaration (Using Loops)

We initialize the array after the declaration by assigning the initial value to each element individually. We can use for loop, while loop, or do-while loop to assign the value to each element of the array.

```
for (int i = 0; i < N; i++) {  
    array_name[i] = valuei;  
}
```

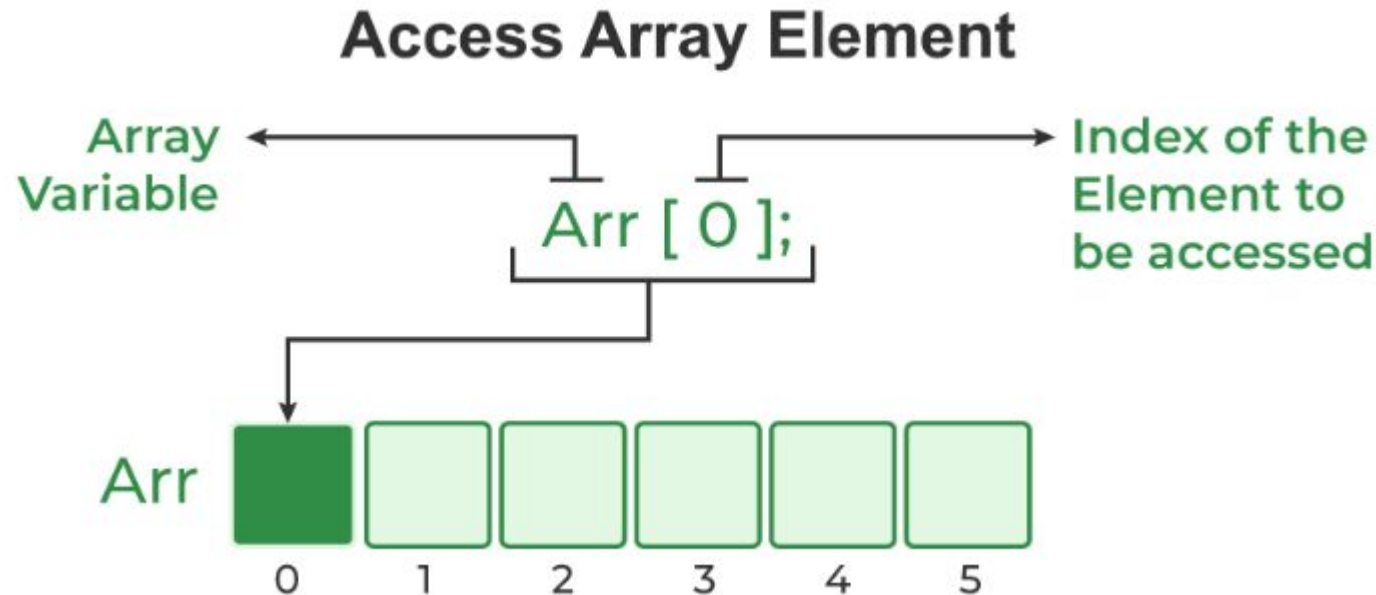
```
// C Program to demonstrate array initialization  
#include <stdio.h>  
  
int main()  
{  
  
    // array initialization using initializer list  
    int arr[5] = { 10, 20, 30, 40, 50 };  
  
    // array initialization using initializer list without  
    // specifying size  
    int arr1[] = { 1, 2, 3, 4, 5 };  
  
    // array initialization using for loop  
    float arr2[5];  
    for (int i = 0; i < 5; i++) {  
        arr2[i] = (float)i * 2.1;  
    }  
    return 0;  
}
```

Access Array Elements

- We can access any element of an array in C using the array subscript operator [] and the index value i of the element.

`array_name [index];`

- Indexing in the array always starts with 0, i.e., the first element is at index 0 and the last element is at $N - 1$ where N is the number of elements in the array.



Example of Accessing Array Elements using Array Subscript Operator

```
// C Program to illustrate element access using array
// subscript
#include <stdio.h>

int main()
{
    // array declaration and initialization
    int arr[5] = { 15, 25, 35, 45, 55 };

    // accessing element at index 2 i.e 3rd element
    printf("Element at arr[2]: %d\n", arr[2]);

    // accessing element at index 4 i.e last element
    printf("Element at arr[4]: %d\n", arr[4]);

    // accessing element at index 0 i.e first element
    printf("Element at arr[0]: %d", arr[0]);

    return 0;
}
```

Output

```
Element at arr[2]: 35
Element at arr[4]: 55
Element at arr[0]: 15
```

```
#include <stdio.h>

int main () {
int num[ 10 ]; /* num is an array of 10 integers */
int i,j; /* initialize elements of array num to 0 */
for ( i = 0; i < 10; i++ )
    { num[ i ] = i + 100; /* set element at location i to i + 100 */ }
for (j = 0; j < 10; j++ ) /* output each array element's value */
    { printf("Element[%d] = %d\n", j, num[j] ); }
return 0; }
```

C Program to Copy all elements of an array into Another array

```
#include<stdio.h>

int main() {
    int arr1[30], arr2[30], i, num;

    printf("\nEnter no of elements :");
    scanf("%d", &num);

    //Accepting values into Array
    printf("\nEnter the values :");
    for (i = 0; i < num; i++) {
        scanf("%d", &arr1[i]);
    }

    /* Copying data from array 'a' to array 'b */
    for (i = 0; i < num; i++) {
        arr2[i] = arr1[i];
    }

    //Printing of all elements of array
    printf("The copied array is :");
    for (i = 0; i < num; i++)
        printf("\narr2[%d] = %d", i, arr2[i]);

    return (0);
}
```

C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe

Enter no of elements :4

Enter the values :6 5 4 3

The copied array is :

arr2[0] = 6

arr2[1] = 5

arr2[2] = 4

arr2[3] = 3

Process exited with return value 0

Press any key to continue . . .

C Program for Reversing an Array Elements

```
#include<stdio.h>
int main() {
    int arr[30], i, j, num, temp;
    printf("\nEnter no of elements : ");
    scanf("%d", &num);
    //Read elements in an array
    for (i = 0; i < num; i++) {
        scanf("%d", &arr[i]);
    }
    j = i - 1;    // j will Point to last Element
    i = 0;        // i will be pointing to first element
    while (i < j) {
        temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
        i++;      // increment i
        j--;      // decrement j
    }
    //Print out the Result of Insertion
    printf("\nResult after reversal : ");
    for (i = 0; i < num; i++) {
        printf("%d \t", arr[i]);
    }
    return (0);
}
```

C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe

```
Enter no of elements : 5
11 22 33 44 55

Result after reversal : 55      44      33      22      11
-----
Process exited with return value 0
Press any key to continue . . .
```


C Program to Calculate Addition of All Elements in Array

```
#include<stdio.h>
int main() {
    int i, arr[50], sum, num;
    printf("\nEnter no of elements :");
    scanf("%d", &num);
    //Reading values into Array
    printf("\nEnter the values :");
    for (i = 0; i < num; i++)
        scanf("%d", &arr[i]);
    //Computation of total
    sum = 0;
    for (i = 0; i < num; i++)
        sum = sum + arr[i];
    //Printing of all elements of array
    for (i = 0; i < num; i++)
        printf("\na[%d]=%d", i, arr[i]);
    //Printing of total
    printf("\nSum=%d", sum);
    return (0);
}
```

C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe

Enter no of elements :4

Enter the values :11 22 33 44

a[0]=11

a[1]=22

a[2]=33

a[3]=44

Sum=110

Process exited with return value 0
Press any key to continue . . .

C Program to Search an element in Array

```
#include<stdio.h>
int main() {
    int a[30], ele, num, i;
    printf("\nEnter no of elements :");
    scanf("%d", &num);
    printf("\nEnter the values :");
    for (i = 0; i < num; i++) {
        scanf("%d", &a[i]);
    }
    //Read the element to be searched
    printf("\nEnter the elements to be searched :");
    scanf("%d", &ele);
    //Search starts from the zeroth location
    i = 0;
    while (i < num && ele != a[i]) {
        i++;
    }
    //If i < num then Match found
    if (i < num) {
        printf("Number found at the location = %d", i + 1);
    } else {
        printf("Number not found");
    }
    return (0);
}
```

C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe

Enter no of elements :5

Enter the values :9 5 7 2 4

Enter the elements to be searched :7

Number found at the location = 3

Process exited with return value 0

Press any key to continue . . .

Two-Dimensional Arrays

- The simplest form of multidimensional array is the two-dimensional array.
- A two-dimensional array is a list of one-dimensional arrays.

Declaration of a 2D Array:

Syntax : `type Arrname[x][y];`

Where type can be any valid C data type and Arrname will be a valid C identifier.

A two-dimensional array consist of x number of rows and y number of columns.

For eg: `int x[2][3];`

Two-Dimensional Arrays

- **Initializing Arrays**

- `int x[2][3]={`
 `{1,2,3}, /*Initializes for row indexed by 0 */`
 `{4,5,6} /*Initializes for row indexed by 1 */`
 `};`

- The nested braces are optional:

`int x[2][3]={1,2,3,4,5,6};`

<code>x[0][0]=1</code>	<code>x[0][1]=2</code>	<code>x[0][2]=3</code>
<code>x[1][0]=4</code>	<code>x[1][1]=5</code>	<code>x[1][2]=6</code>

Two-Dimensional Arrays

- **Accessing Two-Dimensional Array Elements**
- `int number= x[1][2];`
- This statement will take the 3rd element from the 2nd row of the array and assign the value to number variable.
- A nested loop is to be used to handle a two-dimensional array.

C Program to Add Two Matrices

```
#include <stdio.h>
int main() {
    int r, c, a[100][100], b[100][100], sum[100][100], i, j;
    printf("Enter the number of rows (between 1 and 100): ");
    scanf("%d", &r);
    printf("Enter the number of columns (between 1 and 100): ");
    scanf("%d", &c);
    printf("\nEnter elements of 1st matrix:\n");
    for (i = 0; i < r; ++i)
        for (j = 0; j < c; ++j) {
            printf("Enter element a%d%d: ", i + 1, j + 1);
            scanf("%d", &a[i][j]);
        }
    printf("Enter elements of 2nd matrix:\n");
    for (i = 0; i < r; ++i)
        for (j = 0; j < c; ++j) {
            printf("Enter element b%d%d: ", i + 1, j + 1);
            scanf("%d", &b[i][j]);
        }
    // adding two matrices
    for (i = 0; i < r; ++i)
        for (j = 0; j < c; ++j) {
            sum[i][j] = a[i][j] + b[i][j];
        }
    // printing the result
    printf("\nSum of two matrices: \n");
    for (i = 0; i < r; ++i)
        for (j = 0; j < c; ++j) {
            printf("%d\t", sum[i][j]);
            if (j == c - 1) {
                printf("\n\n");
            }
        }
    return 0;
}
```

C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe

Enter the number of rows (between 1 and 100): 2
Enter the number of columns (between 1 and 100): 2

Enter elements of 1st matrix:

Enter element a11: 3

Enter element a12: 4

Enter element a21: 5

Enter element a22: 6

Enter elements of 2nd matrix:

Enter element b11: 1

Enter element b12: 2

Enter element b21: 3

Enter element b22: 4

Sum of two matrices:

4 6

8 10

Process exited with return value 0

Press any key to continue

Activate

Task

- WAP to initialize the matrix and display the matrix.
- WAP to take input as number of rows, column and the elements of the matrix. Display the user given matrix.
- Transpose of a matrix
- Subtraction of two matrices.
- Product of two matrices.

Strings

- It is a collection of characters in a linear sequence.
- It is terminated by a null character '\0'.
- String Declaration:
 - 1. Syntax: `char variableName[number of characters];`
For eg: `char name[6];`
 - 2. Pointers: `char *name;`
- String Initialization:
 - `char name[6]={'I', 'N', 'D', 'I', 'A', '\0'};`
 - `char name[ ]='INDIA';` It will need 6 byte of space

String Functions

- C supports a wide range of functions that manipulate null-terminated strings
- In order to use the in-built string functions, string.h header file needs to be included in the C program
- **strcpy(s1, s2);** // Copies string s2 into string s1.
- **strcat(s1, s2);** // Concatenates string s2 onto the end of string s1.
- **strlen(s1);** // Returns the length of string s1.
- **strcmp(s1, s2);** // This function compares the ASCII Value. Returns 0 if s1 and s2 are the same; returns less than 0 if s1<s2; returns greater than 0 if s1>s2.

Program using String functions

```
#include <stdio.h>
#include <string.h>
int main ()
{
    char college[12] = "SIT";
    char city[12] = "NAGPUR";
    char dummy[12];
    int len, comp ;
    /* copy college into dummy */
    strcpy(dummy, college);
    printf("strcpy( dummy, college) : %s\n", dummy );
    /* concatenates str1 and str2 */
    strcat( college, city);
    printf("strcat( college, city): %s\n", college );
    /* total length of str1 after concatenation */
    len = strlen(college);
    printf("strlen(college) : %d\n", len );
    comp=strcmp(college,city);
    printf("strcmp(college,city) : %d\n", comp );
    return 0;
}
```

C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe

```
strcpy( dummy, college) : SIT
strcat( college, city): SITNAGPUR
strlen(college) : 9
strcmp(college,city) : 1
```

```
-----
Process exited with return value 0
Press any key to continue . . .
```